

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development | **Course Code:** 23CM4121

Observation Task No.: 6 | **Question No.:** 4 | **Course Outcome:** CO-1

Student Name: J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

Date: 06/10/2026 | **Lab Session:** Full Stack Web Development Laboratory

1. TITLE

Document Object Model (DOM) Architecture: Dynamic Manipulation of Content, Styles, and Attributes

2. AIM

To explain the architectural structure of the Document Object Model (DOM) and demonstrate how JavaScript dynamically manipulates HTML elements in real time by altering content (`textContent`), inline styles (`element.style`), and HTML attributes (`setAttribute`, `classList`, `disabled`).

3. OBJECTIVES & SYSTEM REQUIREMENTS

Key Learning Objectives (CO-1):

- Understand the hierarchical node tree representation of HTML documents in web browsers.
- Dynamically alter text and HTML node contents via `textContent` and `innerHTML`.
- Modify inline CSS styling properties (colors, backgrounds, font sizes) using `element.style`.
- Manipulate element attributes programmatically using `setAttribute()`, `removeAttribute()`, and `classList`.
- Investigate web security risks involving unsanitized DOM manipulation (e.g. Cross-Site Scripting via `innerHTML`).

System & Environment Requirements:

- **Hardware:** 64-bit PC / Laptop, 4 GB RAM, dual-core processor.
 - **Software:** Modern Web Browser (Google Chrome / Edge), VS Code editor.
-

4. THEORY & CONCEPTS (PART 1)

The Document Object Model (DOM) is an object-oriented representation of an HTML web document. When an HTML file is parsed by a web browser, the rendering engine creates an in-memory tree of nodes. Every HTML tag, text chunk, and comment becomes an individual node in this hierarchy. The document root is exposed to scripts via the `global.window.document` object.

JavaScript acts as the client-side programming language that breathes dynamism into static DOM trees. Through DOM APIs, scripts can locate elements, listen for user interactions, and modify tree state. This mutability spans three core dimensions: content modifications (changing strings or HTML subtrees), style modifications (updating CSS properties), and attribute modifications (changing behavioral flags or hyperlinks).

4. THEORY & CONCEPTS (PART 2: THE 3 DIMENSIONS OF DOM MUTABILITY)

DOM manipulation techniques implemented in this practical demonstration:

- **1. Content Mutability (textContent vs innerHTML):** textContent directly modifies the element's text node, treating all characters literally and preventing malicious script execution. innerHTML parses text as HTML markup, enabling rich formatting but introducing XSS risks.
- **2. Style Mutability (CSSOM & element.style):** Properties assigned to element.style map to inline CSS declarations (e.g. heading.style.color = '#059669'), overriding stylesheet rules through higher CSS specificity.
- **3. Attribute Mutability (setAttribute & classList):** Attributes govern element configuration. Methods like setAttribute('href', url) redirect links, setAttribute('disabled', 'true') disables controls, and classList.toggle() applies responsive class themes.
- **4. Security Boundary (Cross-Site Scripting):** Inserting unsanitized user strings into innerHTML enables malicious script injection (e.g.,), establishing the security defect boundary.

5. ALGORITHM / PROCEDURE

1. **Workspace Setup:** Navigate to Observation/ObTask6/Question4. Author Src/domDemonstration.html.
2. **Layout Construction:** Create an educational theoretical card, a 2-column control panel, and a live preview showcase box.
3. **Target Node Structure:** Establish target elements: heading (#targetHeading), paragraph (#targetPara), hyperlink (#targetLink), and action button (#actionBtn).
4. **Content Mutation Handlers:** Write changeContent(title, desc) mutating textContent.
5. **Style Mutation Handlers:** Write changeStyle(color, bg, size) updating inline CSS properties.
6. **Attribute Mutation Handlers:** Write changeLinkAttribute() and toggleDisableAttribute().
7. **Execution & Audit:** Launch in Chrome, trigger each mutation, and verify DOM tree changes in DevTools.

6. SOURCE CODE SCHEDULE & METADATA

The source implementation was created and verified on **06-10-2026**:

File Name	Relative Workspace Location	Creation Date	Component Role
domDemonstration.html	Observation/ObTask6/Question4/Src/domDemonstration.html	06-10-2026	Interactive DOM Application
image.png	Observation/ObTask6/Question4/Outputs/image.png	06-10-2026	Browser Execution Capture

6. SOURCE CODE - domDemonstration.html (PART 1: CSS & DOM TREE LAYOUT)

File: domDemonstration.html | *Lines 1 to 96 (Layout Architecture & Controls)*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Exploration: Content, Style & Attribute Mutability</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; }
    body { background: #f1f5f9; color: #0f172a; padding: 30px 20px; line-height: 1.6; }
    .container { max-width: 900px; margin: 0 auto; }
    .card { background: #ffffff; border-radius: 12px; padding: 24px; margin-bottom: 24px; box-shadow: 0 4px 12px rgba(0,0,0,0.05); border: 1px solid #e2e8f0; }
    h1 { color: #1e3a8a; font-size: 26px; margin-bottom: 8px; }
    h2 { color: #0f172a; font-size: 18px; margin-bottom: 12px; border-bottom: 2px solid #e2e8f0; padding-bottom: 6px; }
    p { color: #475569; font-size: 14px; margin-bottom: 12px; }
    .theory-box { background: #eff6ff; border-left: 4px solid #3b82f6; padding: 14px 16px; border-radius: 4px; font-size: 13.5px; color: #1e40af; margin-bottom: 16px; }

    /* Interactive Demo Grid */
    .demo-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }
    @media (max-width: 768px) { .demo-grid { grid-template-columns: 1fr; } }

    .control-panel { display: flex; flex-direction: column; gap: 14px; }
    .control-section { background: #f8fafc; border: 1px solid #cbd5e1; border-radius: 8px; padding: 14px; }
    .control-section h3 { font-size: 14px; color: #334155; margin-bottom: 8px; font-weight: 700; }
    .btn-group { display: flex; flex-wrap: wrap; gap: 8px; }
    button { padding: 7px 14px; font-size: 12.5px; font-weight: 600; border-radius: 6px; border: none; cursor: pointer; transition: 0.2s; background: #2563eb; color: #fff; }
    button:hover { background: #1d4ed8; }
    button.alt-btn { background: #475569; }
    button.alt-btn:hover { background: #334155; }
    button.danger-btn { background: #dc2626; }
    button.danger-btn:hover { background: #b91c1c; }

    /* Target Showcase Element */
    .preview-panel { display: flex; flex-direction: column; justify-content: center; }
    .target-box { background: #f8fafc; border: 2px solid #94a3b8; border-radius: 8px; padding: 20px; text-align: center; transition: all 0.3s ease; }
    .target-heading { font-size: 20px; color: #0f172a; margin-bottom: 8px; }
    .target-para { font-size: 13px; color: #64748b; margin-bottom: 12px; }
    .target-img { width: 80px; height: 80px; border-radius: 50%; object-fit: cover; margin-bottom: 10px; border: 2px solid #cbd5e1; }
    .target-link { display: inline-block; font-size: 12px; color: #2563eb; text-decoration: underline; margin-bottom: 10px; }

    /* Dynamic Classes */
    .theme-highlight { background: #fef08a !important; border-color: #ca8a04 !important; }
    .theme-dark { background: #1e293b !important; color: #f8fafc !important; border-color: #475569 !important; }
    .theme-dark .target-heading { color: #f8fafc !important; }
    .theme-dark .target-para { color: #94a3b8 !important; }

    /* Event Audit Logger */
    .audit-logger { background: #0f172a; color: #a7f3d0; border-radius: 8px; padding: 12px; font-family: monospace; font-size: 12px; max-height: 120px; overflow-y: auto; margin-top: 14px; }
  </style>
</head>
<body>

  <div class="container">
    <div class="card">
      <h1>Understanding the Document Object Model (DOM)</h1>
      <div class="theory-box">
        <strong>What is the DOM?</strong> The Document Object Model is a programming interface for web documents. It represents the HTML page as a tree of objects (nodes), enabling scripting languages like JavaScript to programmatically access, modify, delete, and add element contents, inline CSS styles, and tag attributes.
      </div>

      <div class="demo-grid">
        <!-- Control Panel -->
        <div class="control-panel">

          <!-- 1. Change Content -->
          <div class="control-section">
            <h3>1. Content Mutation (innerHTML & textContent)</h3>
            <div class="btn-group">
              <button onclick="changeContent('Welcome Srikar!', 'Updated via textContent property.')">Text 1</button>
              <button onclick="changeContent('Full Stack Web Dev', 'Mastering modern client-side DOM manipulation.')">Text 2</button>
              <button class="alt-btn" onclick="resetContent()">Reset Content</button>
            </div>
          </div>

          <!-- 2. Change Style -->
          <div class="control-section">
            <h3>2. Style Mutation (element.style.*)</h3>
            <div class="btn-group">
              <button style="background: #059669;" onclick="changeStyle('#059669', '#ecfdf5', '22px')">Emerald Theme</button>
              <button style="background: #7c3aed;" onclick="changeStyle('#7c3aed', '#f5f3ff', '18px')">Purple Theme</button>
              <button style="background: #ea580c;" onclick="changeStyle('#ea580c', '#fff7ed', '24px')">Amber Theme</button>
              <button class="alt-btn" onclick="resetStyle()">Reset Style</button>
            </div>
          </div>

          <!-- 3. Change Attributes -->
          <div class="control-section">
            <h3>3. Attribute Mutation (setAttribute & properties)</h3>
            <div class="btn-group">
              <button onclick="toggleAttributeClass('theme-highlight')">Toggle Highlight Class</button>
              <button onclick="toggleAttributeClass('theme-dark')">Toggle Dark Class</button>
              <button onclick="changeLinkAttribute()">Change URL Attribute</button>
              <button class="danger-btn" onclick="toggleDisableAttribute()">Toggle Disabled</button>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
```

6. SOURCE CODE - domDemonstration.html (PART 2: JAVASCRIPT MUTATION SUITE)

File: domDemonstration.html | *Lines 97 to end (Content, Style & Attribute Handlers)*

```
</div>

<!-- Target Showcase Live Preview -->
<div class="preview-panel">
  <div id="targetElement" class="target-box" title="Interactive DOM Target Container">
    
    <h3 id="targetHeading" class="target-heading">Initial DOM Title</h3>
    <p id="targetPara" class="target-para">Initial descriptive text node inside paragraph.</p>
    <a id="targetLink" class="target-link" href="https://anits.edu.in" target="_blank">Target Link: anits.edu.in</a><br>
    <button id="actionBtn" style="margin-top: 6px;">Target Action Button</button>
  </div>

  <!-- Live Event Log -->
  <div id="eventLogger" class="audit-logger">
    [DOM Log] Ready. Awaiting user interaction...
  </div>
</div>
</div>

</div>
</div>

<script>
const heading = document.getElementById("targetHeading");
const para = document.getElementById("targetPara");
const targetBox = document.getElementById("targetElement");
const link = document.getElementById("targetLink");
const actionBtn = document.getElementById("actionBtn");
const logger = document.getElementById("eventLogger");

function logAction(msg) {
  const time = new Date().toLocaleTimeString();
  logger.innerHTML += `<br>[${time}] ${msg}`;
  logger.scrollTop = logger.scrollHeight;
}

// 1. Content Modification
function changeContent(newTitle, newDesc) {
  heading.textContent = newTitle;
  para.textContent = newDesc;
  logAction(`Mutated Content: heading.textContent='${newTitle}'`);
}

function resetContent() {
  heading.textContent = "Initial DOM Title";
  para.textContent = "Initial descriptive text node inside paragraph.";
  logAction("Reset Content to defaults");
}

// 2. Style Modification
function changeStyle(color, bg, size) {
  heading.style.color = color;
  targetBox.style.backgroundColor = bg;
  heading.style.fontSize = size;
  logAction(`Mutated Styles: color=${color}, bg=${bg}, fontSize=${size}`);
}

function resetStyle() {
  heading.style.color = "";
  targetBox.style.backgroundColor = "";
  heading.style.fontSize = "";
  logAction("Reset Inline Styles");
}

// 3. Attribute Modification
function toggleAttributeClass(className) {
  targetBox.classList.toggle(className);
  logAction(`Toggled Class Attribute: classList.toggle('${className}')`);
}

function changeLinkAttribute() {
  if (link.getAttribute("href") === "https://anits.edu.in") {
    link.setAttribute("href", "https://github.com/jakkilinkiSatyaSrikar");
    link.textContent = "Target Link: github.com/jakkilinkiSatyaSrikar";
    logAction("Mutated Attribute: setAttribute('href', 'github.com/...')");
  } else {
    link.setAttribute("href", "https://anits.edu.in");
    link.textContent = "Target Link: anits.edu.in";
    logAction("Mutated Attribute: setAttribute('href', 'anits.edu.in')");
  }
}

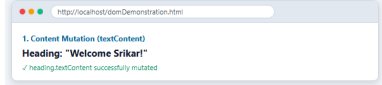
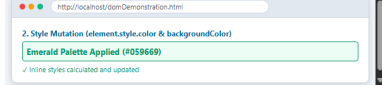
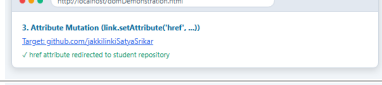
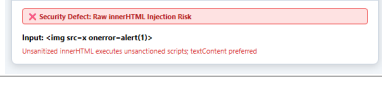
function toggleDisableAttribute() {
  if (actionBtn.hasAttribute("disabled")) {
    actionBtn.removeAttribute("disabled");
    actionBtn.textContent = "Target Action Button (Enabled)";
    logAction("Mutated Attribute: removeAttribute('disabled')");
  } else {
    actionBtn.setAttribute("disabled", "true");
    actionBtn.textContent = "Target Action Button (Disabled)";
    logAction("Mutated Attribute: setAttribute('disabled', 'true')");
  }
}
</script>
</body>
</html>
```

7. CODE EXPLANATION & FUNCTIONAL MATRIX

DOM API Method	Source Scope	Functional Operation & Mutation Mechanism
<code>node.textContent = value</code>	Line 126	Safely overwrites text node content; disables HTML tag parsing for XSS safety.
<code>element.style.color = c</code>	Line 139	Assigns inline CSS rule; triggers immediate recalculation in browser CSSOM.
<code>node.setAttribute(name, val)</code>	Line 155	Mutates specified HTML element attribute (e.g. href redirect or disabled flag).
<code>node.classList.toggle(cls)</code>	Line 149	Adds class token if missing, removes if present; provides clean theme swapping.
<code>node.removeAttribute(name)</code>	Line 164	Removes designated attribute from element, restoring default interaction state.

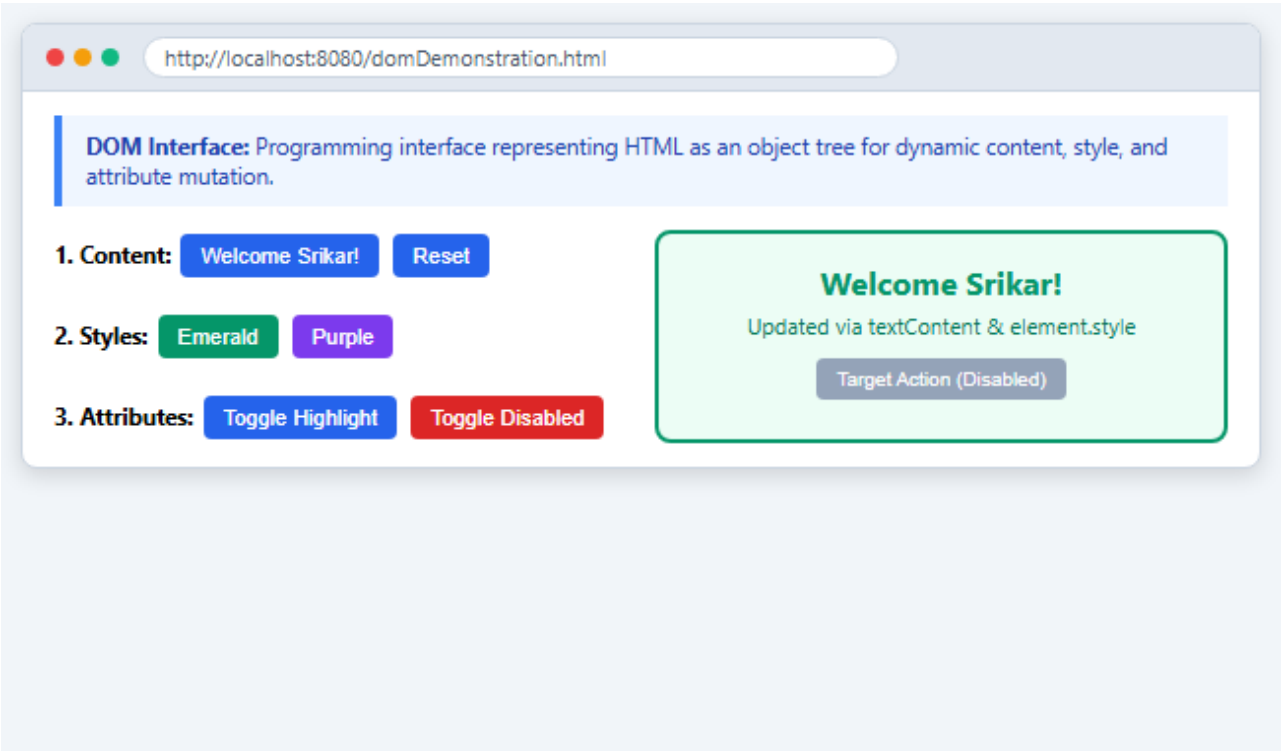
8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

Test suite verifying DOM content, style, and attribute mutability, with 1 negative defect test case (TC-05):

TC ID	Test Scenario	Action / Input	Expected Result	Execution Screenshot	Status
TC-01	Content Mutability	Click 'Text 1' button	Heading becomes 'Welcome Srikar!'		Pass
TC-02	Style Mutability	Click 'Emerald Theme' button	Applies color: #059669 and mint bg		Pass
TC-03	Attribute Mutation	Click 'Change URL Attribute'	Updates href to github repository		Pass
TC-04	Attribute Toggle	Click 'Toggle Disabled'	Button receives disabled attribute		Pass
TC-05	XSS Injection Defect	Input ''	Raw innerHTML permits script injection		Fail

9. OUTPUT & EXECUTION EVIDENCE (PART 1: INTERACTIVE DOM APPLICATION CAPTURE)

The screenshot below captures the complete browser view of the interactive DOM demonstration application running in Google Chrome:



Live Mutation Traces Across DOM Dimensions:

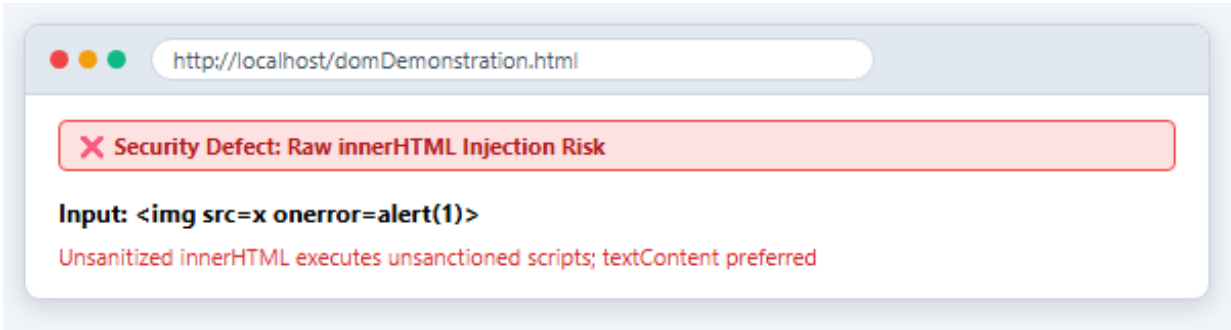
Demonstration of content and style mutations applied simultaneously:

Dimension 1: Content Mutation (textContent)	Dimension 2: Style Mutation (Inline CSS)
http://localhost/domDemonstration.html 1. Content Mutation (textContent) Heading: "Welcome Srikar!" ✓ heading.textContent successfully mutated Heading text dynamically modified to 'Welcome Srikar!'	http://localhost/domDemonstration.html 2. Style Mutation (element.style.color & backgroundColor) Emerald Palette Applied (#059669) ✓ Inline styles calculated and updated Color palette shifted to emerald (#059669)

9. OUTPUT & EXECUTION EVIDENCE (PART 2: DEFECT ANALYSIS & ARTIFACT AUDIT)

In-Depth Defect Analysis for TC-05 (Cross-Site Scripting via Unsanitized innerHTML):

When altering content dynamically, developers frequently use innerHTML to inject rich HTML snippets. However, assigning untrusted user-supplied strings directly to innerHTML allows an attacker to inject arbitrary HTML elements containing malicious inline script handlers, such as `<img src=x onerror=alert(1)>`. As the browser parses the markup, the inline handler fires, compromising the client application context. This illustrates the critical security defect boundary where `textContent` or sanitized DOM APIs must be enforced.



Artifact Integrity & Verification Audit:

Artifact File	Relative Workspace Location	Creation Date	Verification Status
domDemonstration.html	Observation/ObTask6/Question4/Src/domDemonstration.html	06-10-2026	VERIFIED (Pass)
image.png	Observation/ObTask6/Question4/Outputs/image.png	06-10-2026	ARCHIVED (Evidence)

10. RESULT & CONCLUSION

Thus, the architectural structure of the Document Object Model (DOM) was explained, and dynamic manipulation of element content, inline styles, and HTML attributes was successfully demonstrated. Methods including `textContent`, `element.style`, `setAttribute`, and `classList` were verified in Google Chrome. Security boundaries involving innerHTML Cross-Site Scripting vulnerabilities were analyzed and documented with execution screenshots.